Rapport de Projet : Audit Système et Automatisation

Partie 1 : Audit Système

Le but de cette première partie est de décrire toutes les commandes nécessaires a la suite du projet.

Comment ai-je procéder pour trouver les commandes que je ne connaissais pas ?

N'étant pas super familier avec Linux de base, il y a de nombreuses commandes que je ne connaissais pas, avant de chercher toutes commandes, j'ai donc mit en place une "stratégie" qui était de chercher avec `apropos` un mot clé, et ensuite de chercher dans `man` comment l'utiliser et donc comment trouver l'information que je voulais. Par exemple pour la version du noyau, j'ai tapé `apropos kernel` et j'ai vu qu'il y avait `proc_version` qui me permettait de voir la version du noyau. J'ai donc tapé `man proc_version` j'ai vu que je pouvais faire `cat /proc/version`. J'avais donc mon information et cela m'a permis de mettre en place un réflexe de chercher par moi même au lieu d'aller directement chercher sur google (qui est ma solution de secours si je ne trouve vraiment rien).

Nom de la machine

La commande est : `hostname`. Elle retourne "debian-noa". Il n'y a pas besoin de sudo.

Distribution (nom et version)

Nous pouvons le faire avec `cat /etc/os-release` qui donne toutes les informations dont celle que l'on cherche :

- PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
- NAME="Debian GNU/Linux"
- VERSION_ID="13"
- VERSION="13 (trixie)"
- VERSION_CODENAME=trixie
- DEBIAN_VERSION_FULL=13.7
- ID=debian

Et nous pouvons également faire ça avec `lsb_release -d` qui nous retourne : "Description: Debian GNU/Linux 13 (trixie)"

On priorisera d'utiliser `/etc/os-release` car comme l'explique le sujet, la commande `lsb_release` dépend du paquet `lsb-release`, absent des installations minimales et de la plupart des images Docker.
Nous n'avons pas besoin de sudo.

Version du noyau

L'explication est détaillé au dessus dans l'exemple de ma stratégie. Cela donne :

```
noa@debian-noa:~/PreSaeNoa$ cat /proc/version Linux version 6.12.113+deb13-amd64 (debian-kernel@lists.debian.org) (x86_64-linux-gnu-gcc-14 (Debian 14.2.0-19) 14.2.0, GNU ld (GNU Binutils for Debian) 2.44) #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (2026-10-15) noa@debian-noa:~/PreSaeNoa$
```

J'ai également testé `uname -r` ou `-r` donne la version du noyau en cours sur le systeme d'exploitation :

Mémoire et swap

Pour la mémoire et swap disponibles et utilisés, on utilise la commande "`free`". Je ne la connaissais pas avant de faire ce projet, je l'ai découverte en testant les commandes décrites dans les commentaires de la partie 1 du sujet. En faisant `man free`, j'ai également découvert que nous pouvions accéder aux informations que l'on cherche en faisant un `cat /proc/meminfo`.

Elle nous retourne ceci :

```
      noa@debian-noa:~/PreSaeNoa$ free -h total utilisé libre partagé tamp/cache disponible Mem:
1,9Gi 1,0Gi 489Mi 584Ki 553Mi 905Mi Échange: 1,1Gi 0B 1,1Gi noa@debian-noa:~/PreSaeNoa$
```

Le paramètre `-h` permet d'automatiquement mettre à l'échelle la plus appropriée pour qu'un humain puisse lire facilement :

- B = octets
- Ki = kibioctets
- Mi = mébioctets
- Gi = gibioctets
- Ti = tébioctets
- Pi = pébioctets

Le total de mem correspond a la quantité totale de mémoire allouée a mon systeme (je suis dans une VM donc j'avais mis 2GO). Et Le total du swap correspond a la taille totale de l'espace de secours réservé sur mon disque dur de ma VM.
Utilisé correspond a la mémoire utilisé ou a la mémoire indisponible.
Libre correspond a la mémoire inutilisée.
Partagé correspond a la mémoire utilisé surtout par tmpfs.
tamp/cache correspond a la mémoire utilisé par les tampons du noyau, par les cache de page et les slabs.
disponible correspond a la mémoire disponible pour le démarage de nouvelles applications sans utiliser l'espace d'échange.
La différence entre libre et disponible est que la mémoire disponible est l'addition de la mémoire libre et de la mémoire temporairement utilisée car elle peut être récupéré s'il le faut.

Concernant le swap (échange), j'ai fait des recherches sur google cette fois ci, et j'ai vu qu'il n'avait tout simplement pas de système de cache et donc pas colonne disponible.
Nous n'avons pas eu besoin de sudo.

Interfaces réseau

Liste des interfaces réseau : adresses MAC et IP (v4 et v6) associées

```
      noa@debian-noa:~/PreSaeNoa$ ip addr show 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc
noqueue state UNKNOWN group default qlen 1000 link/loopback 00:00:00:00:00:00 brd
00:00:00:00:00:00 inet 127.0.0.1/8 scope host lo valid_lft forever preferred_lft forever inet6
::1/128 scope host noprefixroute valid_lft forever preferred_lft forever 2: enp0s3:
<BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 08:00:27:12:33:59 brd ff:ff:ff:ff:ff:ff altname enx080027123359 inet 10.0.2.15/24 brd
10.0.2.255 scope global dynamic noprefixroute enp0s3 valid_lft 74276sec preferred_lft 63476sec
inet6 fd17:625c:f037:2:a00:27ff:fe12:3359/64 scope global dynamic mngtmpaddr proto kernel_ra
valid_lft 86268sec preferred_lft 14268sec inet6 fd17:625c:f037:2:d099:1513:c9ed:e364/64 scope
global dynamic mngtmpaddr noprefixroute valid_lft 86268sec preferred_lft 14268sec inet6
fe80::bca9:2629:d9c3:f906/64 scope link valid_lft forever preferred_lft forever 3: docker0: <NO-
CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default link/ether
02:42:75:98:93:e7 brd ff:ff:ff:ff:ff:ff inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
```

Utilisateurs humains existants

Liste des utilisateurs humains existants, en distinguant ceux actuellement connectés :
Nous pouvons deja voir tous les comptes de la machine grâce a la commande `cat /etc/passwd`. Ensuite, grâce a la commande `cat /etc/login.defs` (décrite dans le sujet), nous pouvons obtenir l'UID MIN et l'UID MAX, qui sont respectivement 1000 et 60000, c'est à dire que l'uid d'un utilisateur humain ne peut que être compris entre 1000 et 60000, ainsi lorsque nous cherchons dans la sortie de `cat /etc/passwd`, nous pouvons apercevoir qu'il y a que l'utilisateur noa qui est un utilisateur humain :

```
      noa@debian-noa:~$ cat /etc/passwd root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin list:x:38:38:Mailing List
Manager:/var/list:/usr/sbin/nologin irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin systemd-network:x:998:998:systemd
Network Management:/:/usr/sbin/nologin dhcpcd:x:100:65534:DHCP Client
Daemon:/usr/lib/dhcpcd:/bin/false systemd-timesync:x:991:991:systemd Time
Synchronization:/:/usr/sbin/nologin messagebus:x:990:990:System Message
Bus:/nonexistent:/usr/sbin/nologin sshd:x:989:65534:sshd user:/run/sshd:/usr/sbin/nologin
noa:x:1000:1000:noa,,,:/home/noa:/bin/bash noa@debian-noa:~$
```

Nous pouvons également vérifier cela grâce a la commande `who` qui retourne ceci :

```
      noa@debian-noa:~$ who noa sshd pts/0 2026-10-20 21:31 (192.168.1.13) noa sshd 2026-10-20
21:30 (192.168.1.13) noa@debian-noa:~$
```

Ici, il y a deux noa car je suis connecté a la fois via mon VSCODE et à la fois via mon terminal.

Enfin, nous pouvons également voir a partir de la commande `users` qui retourne :

```
      noa@debian-noa:~$ users noa noa noa@debian-noa:~$
```

Nous n'avons pas eu besoin de sudo.

Espace disque par partition

Espace disque par partition (disponible, utilisé) :
Pour obtenir l'espace disque par partition, j'ai encore utilisé une commande qui est dans le sujet : `df`. J'ai lancé `df -h` afin d'avoir comme pour `free -h` des données plus compréhensible pour les espaces libres et utilisé :

```
noa@debian-noa:~/PreSaeNoa$ df -h Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur udev 966M 0 966M 0% /dev tmpfs 198M 560K 197M 1% /run /dev/sda1 7,4G 1,8G 5,3G 25% / tmpfs 987M 0 987M 0% /dev/shm tmpfs 5,0M 0 5,0M 0% /run/lock tmpfs 1,0M 0 1,0M 0% /run/credentials/systemd-journald.service tmpfs 987M 0 987M 0% /tmp /dev/sda6 12G 1,4G 9,2G 14% /home tmpfs 1,0M 0 1,0M 0% /run/credentials/getty@tty1.service tmpfs 198M 4,0K 198M 1% /run/user/1000 noa@debian-noa:~/PreSaeNoa$
```

Mes partitions sont `/dev/sda1` et `/dev/sda6` qui font respectivement 7,4G et 12G.
Pour la partition /dev/sda1, 1,8G sont actuellement utilisés et 5,3G sont disponibles.
Pour la partition /dev/sda6, 1,4G sont utilisés et 9,2G sont disponibles.
Les autres sont des systèmes de fichiers en mémoire (tmpfs), pas des partitions du disque.

J'ai voulu aller plus loin pour chercher si je pouvais avoir uniquement mes deux vraies partitions en sortie, j'ai donc cherché avec `man df`. J'ai trouvé `-t` qui permet de garder uniquement le type que je veux. Je ne connaissais pas encore le type, et juste en dessous de `-t` dans le man, il est écrit que `-T` nous donne le type, j'ai donc lancé `df -T` :

```
noa@debian-noa:~/PreSaeNoa$ df -T Sys. de fichiers Type blocs de 1K Utilisé Disponible Uti% Monté sur udev devtmpfs 989048 0 989048 0% /dev tmpfs tmpfs 202104 560 201544 1% /run /dev/sda1 ext4 7719016 1801724 5503700 25% / tmpfs tmpfs 1010512 0 1010512 0% /dev/shm tmpfs tmpfs 5120 0 5120 0% /run/lock tmpfs tmpfs 1024 0 1024 0% /run/credentials/systemd-journald.service tmpfs tmpfs 1010516 0 1010516 0% /tmp /dev/sda6 ext4 11621552 1436756 9572652 14% /home tmpfs tmpfs 1024 0 1024 0% /run/credentials/getty@tty1.service tmpfs tmpfs 202100 4 202096 1% /run/user/1000 noa@debian-noa:~/PreSaeNoa$
```

Une fois que j'avais mon type, je pouvais trier par type et obtenir ceci :

```
noa@debian-noa:~/PreSaeNoa$ df -t ext4 -h Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur /dev/sda1 7,4G 1,8G 5,3G 25% / /dev/sda6 12G 1,4G 9,2G 14% /home noa@debian-noa:~/PreSaeNoa$
```

On a en sortie uniquement les vraies partition de disque. Je n'ai pas eu besoin de sudo.

Processus consommateurs (CPU et RAM)

Les N processus les plus consommateurs de CPU :
J'ai tout d'abord tapé `apropos process` afin de voir les commandes que je pouvais potentiellement utilisé et j'ai vu qu'il y avait `ps`, c'est la que je me suis rappelé que je pouvais l'utiliser. En fouillant `man ps`, j'ai pu trouver mon %cpu, %mem, comm et mon -o pour le format de sortie.
Nous avons donc les processus trié par ordre décroissant sur %CPU (les 9 premiers). Nous pouvons choisir notre N en ajoutant `-n N` après head et ou N est le nombre que l'on souhaite. Il faut également savoir que si on met par exemple `-n 15`, ça affichera 14 vraies lignes car la première ligne d'en tête compte mais elle n'est pas une vraie ligne.

```
noa@debian-noa:~/PreSaeNoa$ ps -eo pid,%cpu,%mem,comm --sort=-%cpu | head PID %CPU %MEM
COMMAND 1324 1.2 22.3 MainThread 1114 1.0 1.1 code-6d62b1d5da 1129 0.7 6.1 MainThread 1295 0.3 1.1
code-6d62b1d5da 1227 0.3 4.0 MainThread 2636 0.2 0.0 kworker/0:0-ata_sff 1276 0.2 0.3 sshd-session
1204 0.1 6.8 MainThread 3455 0.1 0.0 kworker/0:2-events noa@debian-noa:~/PreSaeNoa$
```

On fait pareil pour les N plus consommateurs de mémoire (N étant paramétrable), mais cette fois ci on trie sur %mem et non sur %cpu.

```
noa@debian-noa:~/PreSaeNoa$ ps -eo pid,%cpu,%mem,comm --sort=-%mem | head PID %CPU %MEM
COMMAND 1324 1.0 22.6 MainThread 1204 0.1 6.8 MainThread 1129 0.4 6.1 MainThread 1170 0.0 4.8
MainThread 1216 0.0 4.2 copilot-runtime 743 0.0 4.2 dockerd 1227 0.2 4.0 MainThread 1388 0.1 3.8
MainThread 1336 0.0 2.9 MainThread noa@debian-noa:~/PreSaeNoa$
```

On peut ajouter `--no-headers` pour enlever les noms de colonnes. Nous n'avons pas eu besoin de sudo.

Connexions réseau

Liste des connexions réseau (TCP et UDP, ports en écoute et connexions établies) avec, pour chacune, le processus associé :
En cherchant dans `man ss`, j'ai trouvé les infos que je voulais :

- `-t` pour TCP
- `-u` pour UDP
- `-a` pour afficher tous les sockets que ce soit en écoute ou pas
- `-p` pour afficher le processus qui utilise le socket
- `-n` pour garder les numéros de ports sans les convertir en noms de services

J'ai donc ma commande qui est `ss -tuapn` et qui affiche ceci :

```
noa@debian-noa:~/PreSaeNoa$ ss -tuapn Netid State Recv-Q Send-Q Local Address:Port Peer
Address:PortProcess udp UNCONN 0 0 10.0.2.15:68 0.0.0.0:* udp UNCONN 0 0
[fd17:625c:f037:2:a00:27ff:fe12:3359]:546 [::]:* udp UNCONN 0 0
[fd17:625c:f037:2:d099:1513:c9ed:e364]:546 [::]:* udp UNCONN 0 0
[fe80::bca9:2629:d9c3:f906]%enp0s3:546 [::]:* tcp LISTEN 0 4096 127.0.0.1:32905 0.0.0.0:* tcp
LISTEN 0 128 127.0.0.1:38001 0.0.0.0:* users:(("code-6d62b1d5da",pid=1295,fd=10)) tcp LISTEN 0 128
0.0.0.0:22 0.0.0.0:* tcp LISTEN 0 128 127.0.0.1:34797 0.0.0.0:* users:(("code-
6d62b1d5da",pid=1114,fd=11)) tcp ESTAB 0 0 10.0.2.15:22 192.168.1.13:51770 tcp ESTAB 0 0
127.0.0.1:41690 127.0.0.1:38001 tcp ESTAB 0 0 127.0.0.1:38001 127.0.0.1:41690 users:(("code-
6d62b1d5da",pid=1295,fd=12)) tcp ESTAB 0 0 10.0.2.15:43930 13.107.5.93:443 users:
(("MainThread",pid=1324,fd=37)) tcp ESTAB 0 0 127.0.0.1:34797 127.0.0.1:52104 users:(("code-
6d62b1d5da",pid=1114,fd=10)) tcp ESTAB 0 0 127.0.0.1:52104 127.0.0.1:34797 users:
(("MainThread",pid=1129,fd=31)) tcp ESTAB 0 0 10.0.2.15:48128 20.199.39.224:443 users:
(("MainThread",pid=1324,fd=27)) tcp ESTAB 0 0 10.0.2.15:48998 20.184.175.12:443 users:
(("MainThread",pid=1324,fd=42)) tcp LISTEN 0 128 [::]:22 [::]:* noa@debian-noa:~/PreSaeNoa$
```

Il y a cependant une limite, c'est que je ne vois que les processus lancé par noa (moi), mais je ne vois pas ceux lancé par root, il faut donc que je rajoute sudo.

A savoir : le sujet demande que le script Ruby devra tourner sans root, donc la ou le processus est vide, il faudra l'indiquer comme "inaccessible" sans planter.

```
      noa@debian-noa:~/PreSaeNoa$ sudo ss -tuapn Netid State Recv-Q Send-Q Local Address:Port Peer
Address:PortProcess udp UNCONN 0 0 10.0.2.15:68 0.0.0.0:* users:(("dhcpcd",pid=968,fd=3)) udp
UNCONN 0 0 [fd17:625c:f037:2:a00:27ff:fe12:3359]:546 [::]:* users:(("dhcpcd",pid=1488,fd=3)) udp
UNCONN 0 0 [fd17:625c:f037:2:d099:1513:c9ed:e364]:546 [::]:* users:(("dhcpcd",pid=929,fd=3)) udp
UNCONN 0 0 [fe80::bca9:2629:d9c3:f906]%enp0s3:546 [::]:* users:(("dhcpcd",pid=804,fd=3)) tcp
LISTEN 0 4096 127.0.0.1:32905 0.0.0.0:* users:(("containerd",pid=724,fd=9)) tcp LISTEN 0 128
127.0.0.1:38001 0.0.0.0:* users:(("code-6d62b1d5da",pid=1295,fd=10)) tcp LISTEN 0 128 0.0.0.0:22
0.0.0.0:* users:(("sshd",pid=709,fd=6)) tcp LISTEN 0 128 127.0.0.1:34797 0.0.0.0:* users:(("code-
6d62b1d5da",pid=1114,fd=11)) tcp ESTAB 0 0 10.0.2.15:22 192.168.1.13:51770 users:(("sshd-
session",pid=1276,fd=7),("sshd-session",pid=1266,fd=7)) tcp ESTAB 0 0 127.0.0.1:41690
127.0.0.1:38001 users:(("sshd-session",pid=1276,fd=11)) tcp ESTAB 0 80 127.0.0.1:38001
127.0.0.1:41690 users:(("code-6d62b1d5da",pid=1295,fd=12)) tcp ESTAB 0 0 127.0.0.1:34797
127.0.0.1:52104 users:(("code-6d62b1d5da",pid=1114,fd=10)) tcp ESTAB 0 0 127.0.0.1:52104
127.0.0.1:34797 users:(("MainThread",pid=1129,fd=31)) tcp LISTEN 0 128 [::]:22 [::]:* users:
(("sshd",pid=709,fd=7)) noa@debian-noa:~/PreSaeNoa$
```

La on a bien tous les processus. On peut également ajouter un H pour éviter d'avoir l'en tête.

Services clés

Présence et statut de certains services clés (ex. : ssh, cron, docker) : actif ou non (active, inactive, failed), activé ou non au démarrage (enabled, disabled) :
Pour ces 3 services, j'ai utilisé les deux mêmes commandes et j'ai juste changé la fin, les deux commandes sont :
`systemctl is-active (ssh/cron/docker)`
`systemctl is-enabled (ssh/cron/docker)`

Chaque commande renvoyait active ou enabled, il n'y a pas eu de inactive ou disabled. Cela siginfie qu'ils sont actuellement lancé et qu'ils seront lancé automatiquement au prochain boot.

Comme le sujet en parle, j'ai également testé `ssh.socket` :

```
      noa@debian-noa:~/PreSaeNoa$ systemctl is-active ssh.socket inactive noa@debian-
noa:~/PreSaeNoa$ systemctl is-enabled ssh.socket disabled noa@debian-noa:~/PreSaeNoa$
```

Cela signifie que `ssh.service` tourne en permanence et écoute sur le port 22 (on peut également le voir notre sudo ss ...)
Je n'ai pas eu besoin de sudo.

Partie 2 : Automatisation avec un script Ruby

Le script s'appelle `audit.rb`. Il regroupe toutes les informations de la partie 1, sans dialogue interactif, et n'utilise que la bibliothèque standard de Ruby : `optparse` pour les options, `json` pour l'export, `open3` pour lancer les commandes et `time` pour la date. Il n'y a aucune gem à installer, ce qui simplifie aussi l'image Docker de la partie 3.

Dépôt git : [URL du dépôt]

Utilisation

```
      ruby audit.rb -f texte # affichage lisible ruby audit.rb # JSON sur la sortie standard ruby
audit.rb -n 10 -s ssh,cron,docker,nginx # 10 processus, services choisis ruby audit.rb -o
audit.json # JSON enregistré dans un fichier ruby audit.rb -r /host -f texte # fichiers système
lus sous /host
```

Option	Rôle	Défaut
`-n`, `--nombre N`	nombre de processus affichés (CPU et mémoire)	5
`-f`, `--format`	`json` ou `texte`	`json`
`-s`, `--services`	liste de services séparés par des virgules	`ssh,cron,docker`
`-r`, `--racine`	répertoire sous lequel lire `/etc`, `/proc` ..	`/`
`-o`, `--sortie`	enregistre le JSON dans ce fichier	aucun

Les valeurs sont vérifiées au lancement : un format inconnu, un `N` inférieur à 1 ou une racine qui n'existe pas arrêtent le script avec un message clair. Les noms de services ne peuvent contenir que des caractères valides pour une unité systemd, puisqu'ils sont passés à `systemctl`.

Organisation du code

Chaque information a sa méthode (`audit_machine`, `audit_uptime`, `audit_memoire`, `audit_reseau`, `audit_utilisateurs`, `audit_disque`, `audit_processus`, `audit_connexions`, `audit_services`). Elle renvoie un Hash, ou un tableau de Hash pour les listes. Le résultat global est un Hash qui contient la date de l'audit, les paramètres utilisés et le retour de chaque méthode. On l'affiche avec `afficher_texte` ou on le convertit avec `JSON.pretty_generate`.

Le JSON a donc ces clés de premier niveau : `date_audit`, `parametres`, `machine`, `uptime_et_charge`, `memoire`, `reseau`, `utilisateurs`, `disques`, `processus_gourmands`, `connexions`, `services_cles`. La date est au format ISO 8601 avec le fuseau (`2026-10-23T14:44:31+00:00`), ce qui permet de comparer sans ambiguïté un audit fait sur l'hôte et un audit fait dans un conteneur.

Ce qui change par rapport à la partie 1

En partie 1, j'ai testé les commandes à la main. Dans le script, certains choix sont différents, pour deux raisons : l'option `--racine` oblige à lire des fichiers plutôt qu'à interroger des commandes, et un script a besoin d'une sortie facile à analyser.

Information	Partie 1	Script	Pourquoi
Nom de la machine	`hostname`	`/etc/hostname` sous la racine	`hostname` interroge le conteneur, pas l'hôte. Si le fichier manque, le script se rabat sur la commande.
Distribution	`cat /etc/os-release`	même fichier, lu avec `PRETTY_NAME` , `NAME` , `VERSION_ID`	pas de dépendance à `lsb-release`
Noyau	`uname -r`	`uname -r`	sortie propre, et c'est la même valeur dans un conteneur (noyau partagé)
Uptime et charge	`uptime`	`/proc/uptime` et `/proc/loadavg`	`uptime` lit ces fichiers et les met en forme. Les lire directement évite d'analyser du texte formaté (le séparateur décimal change avec la langue).
Mémoire et swap	`free -h`	`/proc/meminfo`	même source que `free` . Utilisée = `MemTotal - MemAvailable` , comme `free` .
Réseau	`ip addr show`	`ip -j addr show`	la sortie JSON évite des expressions régulières fragiles
Utilisateurs	`/etc/passwd` , `who` , `users`	`/etc/passwd` filtré par `UID_MIN` et `UID_MAX` , puis sessions logind et `utmp`	voir plus bas
Disques	`df -h`	`df -P` , sur les points de montage de type disque	`-P` donne un format stable, en blocs de 1 Kio
Processus	`ps -eo pid,%cpu,%mem,comm --sort=-%cpu`	même commande avec `--no-headers` , `pcpu` et `pmem`	le script garde les N premières lignes
Connexions	`ss -tuapn`	`ss -H -tuapn`	`-H` supprime l'en-tête, donc une ligne analysée en moins
Services	`systemctl is-active` et `is-enabled`	idem, plus `ssh.socket` quand on teste `ssh`	sur Ubuntu récent, SSH peut être démarré par la socket

Quelques points méritent plus d'explications.

Utilisateurs connectés. `who` et `users` lisent le fichier `utmp` , qui n'existe pas toujours dans un conteneur ou sur un système récent. Le script regarde donc deux sources : les fichiers de session de systemd-logind (`/run/systemd/sessions/` , où chaque fichier contient `USER=` et `STATE=`), et `utmp` via `who <fichier>` . Les chemins passent par la racine (`<racine>/run/...`), et pas par `/var/run` , car `/var/run` est un lien symbolique absolu qui sortirait de la racine choisie.

Utilisateurs humains. On garde les comptes dont l'UID est compris entre `UID_MIN` et `UID_MAX` de

Processus. Le `%CPU` de `ps` est une moyenne depuis le lancement du processus, pas une mesure instantanée. Comme `ruby` vient de démarrer et travaille tout de suite, il arrive souvent en tête (85 % dans notre premier test). Ce n'est pas un défaut du script. Pour une valeur instantanée, il faudrait lire `/proc/stat` deux fois, ou utiliser `top -b -n 2`. Nous avons gardé `ps`, qui suffit pour repérer les gros consommateurs. La colonne `comm` est aussi tronquée à 15 caractères (d'où `jetbrains-toolb`).

Connexions. Sans root, `ss -p` ne montre que les processus de l'utilisateur courant. Le script distingue trois cas. Si le processus est connu, il affiche son nom et son PID. Si l'on n'est pas root et que le processus manque, il écrit « inaccessible sans root ». Si l'on est root et que le processus manque, c'est qu'aucun processus n'est associé au socket, par exemple une connexion en `TIME-WAIT`.

Robustesse

Le sujet impose que le script tourne sans root et ne s'arrête jamais sur une erreur. Voici comment il y répond :

- `executer_commande` lance les commandes sans shell (les arguments sont séparés, donc pas d'injection) avec `LC_ALL=C` pour avoir des sorties non traduites. Elle renvoie la sortie standard, la sortie d'erreur et le code de retour. Une commande absente ou en échec produit un message d'erreur dans le résultat, pas une exception.
- `lire_fichier` renvoie `nil` si un fichier est absent ou illisible, et chaque méthode gère ce cas.
- Les informations qu'on ne peut pas lire sans root sont signalées comme telles.

Partie 3 : Déploiement dans Docker

Le Dockerfile

L'image part de `debian:stable-slim` et installe uniquement ce que le script utilise :

Paquet	Apporte
`ruby`	l'interpréteur, avec `json`, `optparse`, `open3` et `time`
`iproute2`	`ip` et `ss`
`procps`	`ps`
`systemd`	`systemctl`

`uname`, `df`, `who` et `hostname` viennent déjà avec l'image de base. L'installation utilise `--no-install-recommends`, et `/var/lib/apt/lists` est supprimé dans la même couche pour garder l'image légère. Le script est copié dans `/app`. Le point d'entrée est `ENTRYPOINT ["ruby", "/app/audit.rb"]` sous la forme exec : tout ce qui suit le nom de l'image dans `docker run` est transmis au script. Par exemple, `docker run --rm audit-sys -f texte -n 10` lance `ruby /app/audit.rb -f texte -n 10`.

Exécution sans option

```
docker build -t audit-sys . docker run --rm audit-sys -f texte
```

Sans option, le conteneur a ses propres espaces de noms (processus, réseau, nom de machine, système de fichiers) mais partage le noyau de l'hôte. Le résultat est un mélange des deux :

- **Nom de machine** : l'identifiant du conteneur (12 caractères hexadécimaux), car il a son propre espace de noms UTS.
- **Distribution** : celle de l'image (Debian stable), pas celle de l'hôte. Sur un hôte Ubuntu, la différence se verrait tout de suite.
- **Noyau, uptime, charge, mémoire** : ceux de l'hôte. Le noyau est partagé, et `/proc/meminfo` n'est pas virtualisé, donc la RAM totale est celle de la machine entière.
- **Réseau** : seulement `lo` et `eth0`, avec une adresse du réseau `bridge` de Docker (172.17.0.x). Les interfaces `wlp109s0` ou `docker0` de l'hôte ne sont pas visibles.
- **Utilisateurs** : aucun humain. Le `/etc/passwd` de l'image ne contient que des comptes système, donc personne n'est connecté non plus.
- **Disques** : le conteneur n'a pas de partition à lui, sa racine est un système `overlay`. Les seuls montages de type `/dev/` sont les fichiers que Docker y monte (`/etc/hostname`, `/etc/hosts`, `/etc/resolv.conf`), qui viennent du disque de l'hôte.
- **Processus** : seulement `ruby` (PID 1) et la commande `ps` lancée par le script.
- **Connexions** : presque aucune, puisque rien n'écoute dans le conteneur.
- **Services** : `systemctl` ne trouve pas de systemd en fonctionnement ni de socket pour lui parler, donc les services sont signalés comme inactifs ou en erreur.
- **Date** : en UTC.

Pour auditer l'hôte, il faut donc donner au conteneur accès à ce qu'il ne voit pas.

Options retenues pour auditer l'hôte

Le fichier `compose.yaml` regroupe toutes les options. On le lance avec `docker compose run --rm audit`.

Option	Information concernée	Justification
`pid: host`	processus, et `/proc/<pid>` pour `ss -p`	le conteneur partage l'espace de noms PID de l'hôte
`network_mode: host`	interfaces et connexions	`ip` et `ss` voient la vraie pile réseau
`/:/host:ro` avec `--racine /host`	`/etc`, `/proc`, montages	l'hôte entier est visible en lecture seule, et le script lit ses fichiers sous `/host`
`/run/systemd:/run/systemd:ro`	services	contient la socket de systemd et le dossier `system`, que `systemctl` cherche pour savoir si systemd tourne
`SYSTEMD_IGNORE_CHROOT=1`	services	avec `pid: host`, `/proc/1/root` diffère de la racine du conteneur, et `systemctl` se croit dans un chroot et refuse de parler à systemd

Comparaison entre l'hôte et le conteneur

Nous avons lancé le script directement sur l'hôte (Debian 13, noyau 6.12.113, sans root), puis dans le conteneur avec les options ci-dessus.

Information	Hôte (sans root)	Conteneur (avec options)	Explication
Nom, distribution, noyau	`camil`, Debian 13 (trixie), 6.12.113	identique	`/host/etc/hostname` et `/host/etc/os-release` sont les fichiers de l'hôte, et le noyau est partagé
Date	`16:44:26`	`14:44:31+00:00`	même instant : l'hôte est en UTC+2, le conteneur en UTC. Les 5 secondes viennent du délai entre les deux lancements.
Uptime	1158,7 s	1164,1 s	même noyau, les 5,4 s correspondent au délai entre les lancements
RAM utilisée	3 970 656 ko	4 013 716 ko	`/host/proc/meminfo` est celui de l'hôte
Interfaces	8 interfaces	les mêmes, avec les mêmes MAC	`network_mode: host`
Utilisateurs	`camil` connecté	`camil` connecté	lecture des sessions logind sous `/host/run/systemd/sessions`
Disques	`/` et `/boot/efi`	`/` et `/boot/efi`	le préfixe `/host` est retiré à l'affichage. La sortie de l'hôte listait aussi les montages snap, car elle date d'avant le filtre `squashfs`.
Processus gourmands	`ruby` 85,7 %	`ruby` 69,2 % et `docker-compose` 14,2 %	`docker-compose`, le client qui a lancé le conteneur, est visible grâce à `pid: host`. Le `%CPU` est une moyenne depuis le démarrage.
Connexions	10 connexions sur 23 en « inaccessible sans root »	`avahi-daemon`, `NetworkManager` et `cupsd` identifiés	voir ci-dessous
Services	`cron` et `docker` actifs et activés, `ssh` non installé	identique	socket de systemd montée

La ligne des connexions est la seule vraie différence, et elle ne vient pas de Docker. Sur l'hôte, nous étions un utilisateur normal. Dans le conteneur, nous sommes root avec `SYS_PTRACE`, donc `ss -p` voit tous les processus. Sept des dix connexions « inaccessibles » ont été identifiées (4 sockets d'`avahi-daemon`, 1 de `NetworkManager`, 2 de `cupsd`). Les trois autres n'ont pas de propriétaire : deux sont en `TIME-WAIT` et une en `CLOSING`. Ce sont des connexions en cours de fermeture, que le noyau gère seul. Pour comparer à égalité, il faudrait lancer `sudo ruby audit.rb -f texte` sur l'hôte, ce qui doit donner les mêmes processus.

Le conteneur affiche aussi le fuseau horaire et les adresses IPv6, que l'ancienne sortie de l'hôte n'avait pas : c'est une différence de version du script, pas de Docker.

Implications en matière de sécurité

Ces options rendent le conteneur beaucoup moins isolé que d'habitude. Voici ce qu'un attaquant qui prendrait le contrôle de ce conteneur pourrait faire.

`pid: host` **avec** `SYS_PTRACE` **donne accès à la mémoire des processus de l'hôte.** Le conteneur peut lire `/proc/<pid>/mem` et s'attacher aux processus. Avec `apparmor=unconfined`, plus aucun profil ne limite cet accès.

`network_mode: host` **supprime l'isolation réseau.** Le conteneur peut écouter sur les ports de l'hôte, atteindre les services qui n'écoutent que sur `127.0.0.1`, et capturer du trafic puisque la capacité `NET_RAW` fait partie des droits par défaut de Docker.

Les options `no-new-privileges`, `read_only` et l'absence de `--privileged` limitent les dégâts, mais elles ne compensent pas les accès précédents. Pour un audit ponctuel sur sa propre machine, le risque est acceptable. Il ne le serait pas pour un conteneur qui tourne en permanence ou qui expose un service.

Pistes pour réduire les droits

- Monter seulement les fichiers lus (`/etc/os-release`, `/etc/passwd`, `/etc/login.defs`, `/etc/hostname`) au lieu de la racine entière. Le script lit alors `/proc` grâce à `pid: host`.
- Supprimer toutes les capacités avec `cap_drop: ALL`, puis ajouter seulement `SYS_PTRACE` et celles que `ss -p` exige encore. À tester, car certaines lectures de `/proc/<pid>/fd` demandent aussi de contourner les droits d'accès aux fichiers.
- Remplacer `apparmor=unconfined` par un profil AppArmor écrit pour ce conteneur, qui autorise `ptrace` en lecture et rien d'autre.
- Se passer de la socket systemd si l'état des services n'est pas indispensable, ou la remplacer par la lecture des fichiers de `/run/systemd`.
- Pour un audit régulier, lancer le script directement sur l'hôte, par exemple avec `cron`, évite tous ces compromis.

Conclusion

Le script répond à la partie 1 en regroupant les commandes testées à la main, avec quelques adaptations pour qu'il marche aussi dans un conteneur. Le plus dur a été la partie Docker. Trois difficultés ont demandé du temps :

- `systemctl` refusait de répondre tant que nous n'avions pas compris le problème du chroot : avec `pid: host`, il compare `/proc/1/root` à sa propre racine, les trouve différentes, et croit être dans un chroot. Il a fallu `SYSTEMD_IGNORE_CHROOT`.
- `ss -p` n'affichait aucun processus de l'hôte avant l'ajout de `SYS_PTRACE` et de `apparmor=unconfined`.
- `df` listait des dizaines de montages snap et ajoutait le préfixe `/host` dans le conteneur, ce qui nous a conduits à lire la table des montages du processus 1.